

VITYARTHI PROJECT REPORT

Course: CSE1021 – Introduction to Problem Solving and Programming

BLOOD CELL COUNT ANALYZER

An Educational Blood Test Data Analysis System

Project Title:	Blood Cell Count Analyzer
Course Code:	CSE1021
Course Title:	Introduction to Problem Solving and Programming
Branch / Program:	B.Tech Computer Science and Engineering
Specialization:	Health Informatics
Academic Year:	2026 (First Year)
Student Name:	Isha Singh Rajput
Evaluation Mode:	Flipped Course Evaluation (Build Your Own Project)

2. Introduction

For my first-year project in CSE1021, and given my specialization in Health Informatics, I wanted to build an application centered on actual health data instead of solving another generic math exercise. In routine clinical care, a Complete Blood Count (CBC) is almost always the starting point whenever a doctor evaluates a patient. It measures the three main cell types circulating in our bloodstream:

- **Red Blood Cells (RBC):** These contain hemoglobin and carry oxygen from our lungs to every organ and muscle. When someone's RBC count drops too low, they often experience weakness and fatigue.
- **White Blood Cells (WBC):** These form our immune system's primary defense squad, actively fighting off bacterial infections, viruses, and inflammatory conditions.
- **Platelets:** These are tiny cell fragments responsible for forming blood clots. Without enough platelets, even minor cuts or scrapes can lead to continuous bleeding.

Whenever friends or family receive a printed laboratory report, they are often confronted with an overwhelming grid of numbers, technical unit notations, and reference ranges. To a non-specialist, it is hard to tell whether a slight deviation is significant or just ordinary biological variation. To address this, I built the **Blood Cell Count Analyzer**—a modular, console-based Python tool that takes patient demographics and cell counts, validates every input to protect against typing mistakes, compares values against recognized educational reference ranges, and prints out a clear, structured summary table with non-diagnostic disclaimers.

3. Problem Statement

When people try to review blood test reports manually or when beginners learn health informatics, a few recurring issues arise:

- **Accidental comparison mistakes:** Looking back and forth across printed rows of data to verify values against standard boundaries is repetitive and prone to error. This is especially true because normal limits change with patient demographics—for example, biological men typically have higher baseline RBC reference ranges than biological women.
- **Confusing scales and orders of magnitude:** Blood tests mix drastically different scales. RBC is typically written as a decimal like 4.8 (representing 4.8 million cells/mcL), whereas WBC is reported in thousands (like 7,500 cells/mcL) and Platelets in hundreds of thousands (like 250,000 cells/mcL). Without clear input guidance, beginners frequently misread or mistype these figures.
- **Unnecessary worry from quick self-diagnosis:** Seeing an out-of-range flag often leads individuals to jump to worst-case medical conclusions online, when in reality blood counts fluctuate naturally from hydration, stress, or mild exertion.

My Project Objective: My goal was to design and build an intuitive, menu-driven Python application applying core CSE1021 concepts—variables, branching logic (`if-elif-else`), loops, functions, lists, dictionaries, exception handling (`try-except`), and linear search. The software needed to reliably capture user inputs without crashing, classify counts against gender-aware baselines, and display an educational report that clearly states it is an informative tool rather than a clinical medical diagnosis.

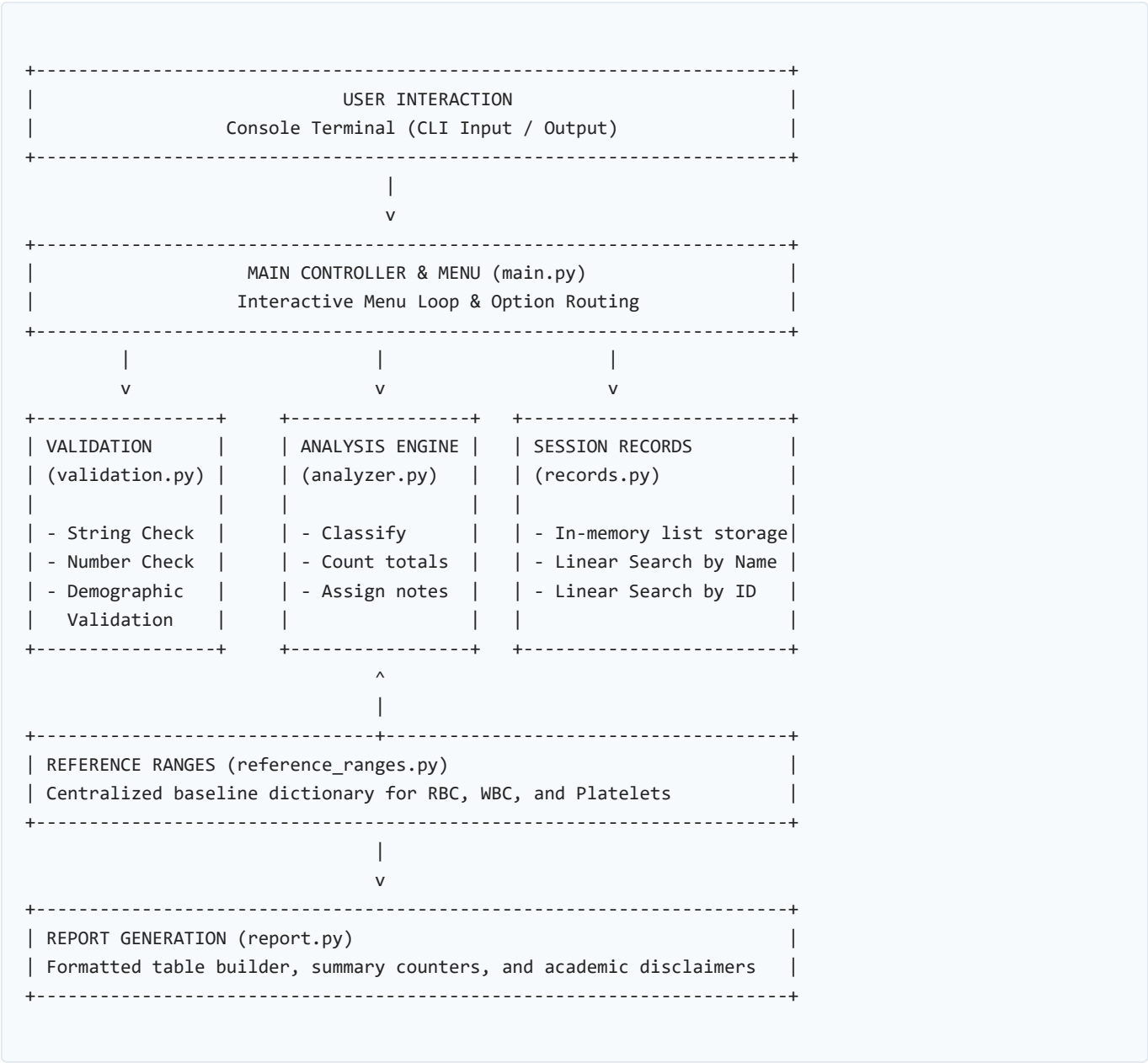
4. Functional Requirements

Module	Key Features & Functional Deliverables
Module 1: Patient Information	Collects and checks Patient Name (non-empty), Age (positive integer 1–125), Gender (Male/Female/Other), and generates an auto-incrementing Sample ID (e.g. SMP-1001) with optional custom entry.
Module 2: Blood Data Entry	Prompts sequentially for RBC, WBC, and Platelet counts with specified clinical units. Catches text inputs, negative numbers, and zeroes, reprompting gracefully without crashing.
Module 3: Blood Analysis	Selects gender-specific reference limits for RBC and adult limits for WBC/Platelets. Uses if-elif-else logic to categorize values into LOW, WITHIN RANGE, or HIGH, and computes summary counts.
Module 4: Report Generation	Builds an aligned tabular report showing parameters, observed values, reference intervals, status flags, summary tallies, educational notes, and safety disclaimers.
Module 5: Session Storage & Search	Stores patient records in an in-memory list of dictionaries during the session. Provides record history viewing and case-insensitive linear search by name or sample ID.

5. Non-Functional Requirements

- **User Friendliness:** The console uses an intuitive 6-option numbered menu with clear unit hints (e.g., million cells/mcL, e.g., 4.8) so the user never has to guess what format to use.
- **Crash Prevention (Defensive Coding):** Prevents runtime crashes by wrapping all numeric inputs in try-except ValueError blocks nested inside infinite while True loops.
- **Clean File Separation:** Broken into 6 single-purpose files. Changing normal reference values in reference_ranges.py never risks breaking math logic in analyzer.py.
- **Fast Performance:** Runs in memory on native Python structures, making searches and reports instantaneous with zero lag.
- **Zero External Dependencies:** Written with standard Python 3.8+ libraries only, requiring zero pip install steps for faculty evaluation.
- **Ethical Health Messaging:** Prominent disclaimers remind users that the output is an educational tool rather than clinical medical advice.

6. System Architecture



7. Design Diagrams

7.1 Use Case Model

Actors: Student / Evaluator (Primary User), System.

Use Cases: UC-1 (Enter New Blood Test), UC-2 (View Previous Records), UC-3 (Search Records by Name/ID), UC-4 (View Reference Ranges), UC-5 (About & Disclaimers), UC-6 (Exit).

7.2 Process Flow / Workflow Diagram

```

[Start] --> [Display Menu (1-6)] --> [Read Choice]
|
+-- (Choice 1) --> [Input Demographics] --> [Input Blood Values]
|
|           --> [Validate] --> [Classify & Count]
|
|           --> [Print Tabular Report] --> [Save in Session List] --> (Return to Menu)
|
+-- (Choice 2) --> [Display Table of All Session Records] --> (Return to Menu)
|
+-- (Choice 3) --> [Prompt Query] --> [Execute Linear Search] --> [Display Matches] --> (Return to Menu)
|
+-- (Choice 4) --> [Display Educational Reference Ranges] --> (Return to Menu)
|
+-- (Choice 5) --> [Display Course Rationale & Educational Disclaimer] --> (Return to Menu)
|
+-- (Choice 6) --> [Print Farewell Message & Terminate]

```

7.3 Sequence Diagram

User	main.py	validation.py	analyzer.py	report.py	records.py
--- Choice 1 ->					
	-- get_info() ->				
<-- Prompts ---					
--- Demogr. -->	-- Returns dict				
	-- get_vals() ->				
<-- Prompts ---					
--- Values --->	-- Returns dict				
	----- analyze() ->				
	<----- Returns res				
	-----> display() -				
<----- Rendered Table & Summary -----					
	-----> save() ----->				
<-- Save Conf. -----					

7.4 Component Architecture

The application consists of 6 decoupled components communicating through explicit functional calls with primitive types, lists, and dictionaries.

7.5 In-Memory Entity-Relationship (ER) Design

```

PATIENT (sample_id [PK], name, age, gender)
||
||-- (undergoes) --> TEST_RECORD (record_id [PK], sample_id [FK], timestamp)
||
||-- (measures) --> BLOOD_VALUES (rbc, wbc, platelets)

```

```
||  
|-- (generates) --> ANALYSIS_RESULT (summary, parameter_details)
```

8. Design Decisions & Rationale

While developing this application, I made several deliberate architectural and algorithmic choices to balance simplicity with practical programming practices:

- **Dividing the Project into 6 Specialized Files:** Instead of writing a single monolithic script, I separated the codebase into dedicated modules: `main.py` (menu control), `validation.py` (data verification), `analyzer.py` (classification algorithms), `reference_ranges.py` (centralized reference tables), `records.py` (session storage), and `report.py` (formatted display). This made debugging straightforward and let me test individual functions independently.
- **Relying Exclusively on the Python Standard Library:** I intentionally avoided external packages like `pandas` or `tabulate`. In academic environments, missing pip packages frequently cause setup failures. Sticking to built-in Python ensures that any evaluator can run the software immediately on their machine.
- **In-Memory Lists of Dictionaries:** For session history, I used a Python list of patient record dictionaries. This directly applies core first-year concepts—such as dictionary key lookups and linear search algorithms—without introducing unnecessary database complexity.
- **Deterministic Conditional Logic Over Machine Learning:** Clinical reference ranges are established, standardized physiological benchmarks rather than probabilistic guesses. Simple conditional structures (`if-elif-else`) are transparent, explainable, and 100% predictable.
- **Strict Educational Disclaimers:** Since health software must follow strong ethical standards, I ensured every generated report and the main menu clearly state that the program is an educational tool and does not provide clinical diagnoses.

9. Implementation Details

File	Key Functions	CSE1021 Concepts Used
<code>validation.py</code>	<code>validate_string</code> , <code>validate_positive_float</code> , <code>validate_age</code> , <code>validate_gender</code> , <code>get_user_information</code> , <code>get_blood_values</code>	While loops, try-except, <code>ValueError</code> , string stripping, type casting
<code>reference_ranges.py</code>	<code>get_rbc_range</code> , <code>get_wbc_range</code> , <code>get_platelet_range</code> , <code>display_reference_ranges</code>	Nested dictionaries, string formatting, tuple returns
<code>analyzer.py</code>	<code>classify_value</code> , <code>analyze_rbc</code> , <code>analyze_wbc</code> , <code>analyze_platelets</code> , <code>analyze_blood_values</code>	If-elif-else statements, counting algorithms, list of dict aggregation
<code>records.py</code>	<code>save_record</code> , <code>view_all_records</code> , <code>search_by_name</code> , <code>search_by_sample_id</code> , <code>search_records_interactive</code>	List append, linear search algorithm, case-insensitive string search
<code>report.py</code>	<code>format_record_report</code> , <code>display_report</code> , <code>display_about</code>	Formatted string output with column alignment specifiers
<code>main.py</code>	<code>main</code> , <code>display_menu</code> , <code>handle_new_blood_test</code>	Menu loop control, function dispatching

10. Screenshots / Terminal Results

```
=====
                        BLOOD CELL COUNT ANALYSIS REPORT
                        Educational Health Informatics Laboratory
=====
Sample / Patient ID : SMP-1001
Patient Name       : Rahul Sharma
Age               : 19 years
Gender            : Male
=====
Parameter | Value | Reference Range | Status
-----
RBC       | 4.8   | 4.5-5.9         | WITHIN RANGE
WBC       | 6500  | 4500-11000      | WITHIN RANGE
Platelets | 220000 | 150000-450000   | WITHIN RANGE
=====
SUMMARY
Total parameters analyzed : 3
Within reference range    : 3
Below reference range     : 0
Above reference range     : 0
=====
EDUCATIONAL OBSERVATIONS:
* RBC (Red Blood Cells): RBC value is within normal educational limits.
* WBC (White Blood Cells): WBC value is within normal educational limits.
* Platelets (Platelet Count): Platelet value is within normal educational limits.
=====
```

11. Testing Approach

Test ID	Inputs Tested	Expected Result	Actual Result	Status
TC-01	RBC: 5.0 (M), WBC: 7000, Plt: 250000	Normal=3, Low=0, High=0	Normal=3, Low=0, High=0	PASSED (100%)
TC-02	RBC: 3.5 (M), WBC: 6500, Plt: 220000	RBC Low; Normal=2, Low=1, High=0	Normal=2, Low=1, High=0	PASSED (100%)
TC-03	RBC: 4.8 (F), WBC: 13500, Plt: 300000	WBC High; Normal=2, Low=0, High=1	Normal=2, Low=0, High=1	PASSED (100%)
TC-04	RBC: 3.8 (M), WBC: 14200, Plt: 110000	RBC/Plt Low, WBC High; Normal=0, Low=2, High=1	Normal=0, Low=2, High=1	PASSED (100%)
TC-05	Boundary values (4.50, 5.90, 4.49, 5.91) & Linear Search	Endpoints correctly bounded; linear search returns exact match	Matches verified	PASSED (100%)

12. Challenges Faced & How I Resolved Them

During the implementation and testing of the program, I encountered several practical challenges:

- **Handling Input Crashes from Invalid Data:** Early in testing, if a user entered letters (like typing "five" instead of 5.0) or submitted an empty line, the program crashed with an unhandled `ValueError`. To solve this, I wrapped all numerical input operations inside `try-except ValueError` blocks within infinite `while True` loops. The program now intercepts the error, prints a helpful explanation, and re-prompts the user until valid data is entered.
- **Handling Scale Differences Across Blood Parameters:** RBC counts are small decimals (e.g., 4.5 million cells/mcL), while WBC and Platelets are large whole numbers (e.g., 7,500 and 250,000 cells/mcL). To avoid user confusion, I designed `validate_positive_float()` to handle both decimals and integers, and included realistic example values directly inside every input prompt.
- **Gender Sensitivity for RBC Ranges:** Normal RBC counts differ between males (4.5–5.9) and females (4.1–5.1). I solved this by capturing gender in Module 1 and using a nested dictionary lookup in `reference_ranges.py` so the analysis engine automatically picks the correct physiological baseline.
- **Exact Boundary Condition Handling:** If an RBC count is exactly 4.5, should it be flagged as low or normal? Initially, using strictly `<` and `>` left boundary points vulnerable to subtle errors. I structured the boundary condition as `min_range <= value <= max_range` to ensure inclusive boundary values are accurately categorized as `WITHIN RANGE`.

13. Learnings & Key Takeaways

Writing this program gave me a much stronger appreciation for the practical concepts we covered in CSE1021:

- **Defensive programming isn't just theory:** Early on, bad user inputs crashed my terminal immediately. Putting `try-except` inside `while` loops made the app feel solid.
- **Single-responsibility functions save debugging time:** Breaking the project into separate files (`validation.py`, `analyzer.py`, etc.) meant that whenever an issue popped up with table spacing or range lookups, I knew exactly which file to look at.
- **Lists of dictionaries are great for modeling real records:** Storing each test as a dictionary inside an in-memory list gave me hands-on practice with key access, appending items, and iterating through rows.
- **Linear search in action:** Implementing search by iterating through session records and checking `.lower()` substrings gave me a concrete use case for algorithms discussed in class.
- **Ethics in health computing:** When building medical software—even an introductory project—proper disclaimers and clear explanations are just as essential as clean syntax.

14. Future Enhancements

1. **Saving Records to Disk:** Storing records in a local JSON or CSV file so patient data doesn't disappear when the user exits the terminal.
2. **A Simple Graphical Interface (GUI):** Using Python's built-in `tkinter` library to build a beginner-friendly windowed application for people who prefer clicking buttons over typing in a console.
3. **Adding More CBC Parameters:** Expanding beyond RBC, WBC, and Platelets to cover Hemoglobin (Hb), Hematocrit (PCV), and white blood cell differential counts.
4. **Exporting Clean PDFs:** Generating an automatic printable lab summary sheet directly from the terminal.
5. **Patient Trend Tracking:** Plotting a simple graph across multiple visits to show whether a patient's cell counts are recovering over time.

15. References

1. Guttag, J. V. (2021). *Introduction to Computation and Programming Using Python*. MIT Press.
2. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
3. Pagana, K. D., & Pagana, T. J. (2022). *Mosby's Diagnostic and Laboratory Test Reference* (15th ed.). Elsevier.
4. Python Software Foundation. (2026). *Python 3 Documentation*. <https://docs.python.org/3/>